

# JavaScript Advanced Concepts - Coding Problems

These coding problems are based on the topics covered in your notes about `this`, `call/apply/bind`, `prototypes`, and `ES6 classes`.

---

## 1 The `this` Keyword

### Problem 1: Global vs Function `this`

Create a function `showThis()` and print the value of `this` when:

- Called normally
- Called in strict mode

#### Expected Learning

- Default Binding
  - Strict Mode Behavior
- 

### Problem 2: Object Method Context

Create an object:

```
const user = {  
  name: "Anubhav"  
};
```

Add a method that prints:

```
Hello Anubhav
```

Then store the method in another variable and call it.

#### Expected Learning

- Method Binding
  - Call Site Concept
- 

## Problem 3: Arrow Function vs Regular Function

Create an object with:

```
name: "Rahul"
```

Implement:

- One regular method
- One arrow method

Print `this.name` from both.

### Expected Learning

- Lexical `this`
  - Object Method Context
- 

## Problem 4: Nested Callback Problem

Create an object:

```
{  
  name: "Rahul",  
  hobbies: ["Coding", "Gaming", "Reading"]  
}
```

Print:

```
Rahul likes Coding  
Rahul likes Gaming  
Rahul likes Reading
```

without storing `this` in another variable.

### Expected Learning

- Arrow Functions
  - Preserving `this`
- 

## Problem 5: Event Handler Simulation

Create an object representing a button.

Write:

- One regular function handler
- One arrow function handler

Compare the value of `this`.

### Expected Learning

- Event Context
  - Lexical Scope
- 

## 2 `call()`, `apply()`, `bind()`

### Problem 6: Borrow a Method using `call()`

Create:

```
const person1 = { name: "Anubhav" };  
const person2 = { name: "Rahul" };
```

Create a method that introduces a person and use `call()` to borrow it.

### Expected Output

```
Hi, I am Anubhav  
Hi, I am Rahul
```

---

### Problem 7: `apply()` with Array Arguments

Create a function:

```
introduce(city, country)
```

Pass values using `apply()`.

## Expected Output

```
I am Rahul from Indore, India
```

## Problem 8: bind() for Delayed Execution

Create a function that prints a user's name after 2 seconds using:

```
setTimeout()
```

and `bind()`.

## Expected Learning

- Permanent Binding
- Callback Context

## Problem 9: Custom Calculator

Create an object:

```
{  
  value: 100  
}
```

Create a function that adds numbers to `value`.

Use:

- `call()`
- `apply()`
- `bind()`

to execute it.

## 3 Prototypes

### Problem 10: Prototype Lookup

Create:

```
const person = {  
  name: "Rahul"  
};
```

Check whether:

```
person.hasOwnProperty("name")
```

comes from the object itself or its prototype.

### Expected Learning

- Prototype Chain

### Problem 11: Create a Custom Prototype Method

Add a method to:

```
Array.prototype
```

called:

```
sum()
```

Example:

```
[1, 2, 3, 4].sum()
```

Output:

```
10
```

## Problem 12: Object.create()

Create:

```
animal
```

object containing:

```
eat()  
sleep()
```

Create a `dog` object using:

```
Object.create()
```

and access inherited methods.

---

## Problem 13: Prototype Inheritance

Create:

```
vehicle
```

object.

Create:

```
car  
bike  
truck
```

using prototype inheritance.

Each should inherit:

```
start()  
stop()
```

---

## Problem 14: Constructor Function + Prototype

Create a constructor:

```
Person
```

that accepts:

```
name  
age
```

Add a method using:

```
Person.prototype.greet
```

## Expected Output

```
Hi, I am Rahul
```

## Problem 15: Prototype Chain Investigation

Create:

```
const arr = [];
```

Print:

```
arr.__proto__  
arr.__proto__.__proto__  
arr.__proto__.__proto__.__proto__
```

Explain the output.

## 4 ES6 Classes

### Problem 16: Basic Class

Create a class:

Student

with:

name  
course

Add a method:

```
introduce()
```

## Expected Output

```
I am Anubhav and I study MERN Stack
```

## Problem 17: Employee Management

Create a class:

Employee

with:

name  
salary

Add methods:

```
increaseSalary()  
showSalary()
```

## Problem 18: Bank Account System

Create a class:



```
BankAccount
```

Features:

- Deposit
- Withdraw
- Check Balance

## Bonus

Prevent withdrawing more than available balance.

---

## Problem 19: Inheritance Challenge

Create:

```
Animal
```

class.

Create:

```
Dog
```

class using:

```
extends
```

Methods:

```
eat()  
bark()
```

## Problem 20: Multi-Level Inheritance

Create:

```
Person  
↓
```

```
Employee
  ↓
Manager
```

Add unique properties and methods at each level.

---

## 5 Static Methods

### Problem 21: Math Utility Class

Create:

```
MathHelper
```

with static methods:

```
add()
subtract()
multiply()
divide()
```

Use them without creating an object.

---

### Problem 22: User Counter

Create a class:

```
User
```

Use a static property to count how many users have been created.

### Example

```
Total Users: 5
```

---

## 6 Getters & Setters

## Problem 23: Full Name Getter

Create:

```
Person
```

with:

```
firstName  
lastName
```

Create a getter:

```
fullName
```

## Problem 24: Email Validation Setter

Create a setter:

```
email
```

Reject invalid emails.

### Example

```
Invalid Email
```

## Private Fields

### Problem 25: Secure Bank Account

Create:

```
#balance
```

as a private field.

Allow:

```
deposit()  
withdraw()  
getBalance()
```

Disallow direct access.

---

## Problem 26: Student Grades System

Create:

```
#marks
```

Store marks privately.

Provide methods:

```
setMarks()  
getMarks()
```

---